

Практическое занятие №8: Машинное обучение с scikit-learn II

Классификация: логистическая регрессия, дерево решений, оценка по метрикам

Цели занятия

1. **Обучающая:** сформировать у студентов практические навыки применения двух ключевых алгоритмов классификации — логистической регрессии и дерева решений — с использованием библиотеки scikit-learn.
2. **Развивающая:** научить студентов выполнять полный цикл классификационного анализа: от загрузки данных и разделения на выборки до построения моделей, оценки их качества и интерпретации результатов.
3. **Воспитательная:** привить культуру объективной оценки моделей машинного обучения с использованием стандартных метрик (accuracy, precision, recall, F1-score) и обоснованного выбора алгоритма в зависимости от структуры данных.

Образовательные результаты: По завершении занятия студент будет уметь:

- Применять логистическую регрессию для решения задач бинарной и многоклассовой классификации.
- Применять деревья решений, понимать принцип их работы и настраивать гиперпараметры для предотвращения переобучения.
- Вычислять и интерпретировать матрицу ошибок (confusion matrix) и метрики precision, recall, F1-score.
- Сравнивать качество нескольких классификаторов и выбирать лучший для конкретной задачи.

Задачи

1. Загрузить датасет и выполнить первичный анализ.
2. Разделить данные на признаки (x) и целевую переменную (y).
3. Разбить данные на обучающую и тестовую выборки с фиксацией `random_state`.
4. Обучить модель **логистической регрессии** и оценить её качество.
5. Обучить модель **дерева решений** с подбором гиперпараметров (ограничение глубины и минимального числа образцов для разбиения с целью борьбы с переобучением).
6. Вычислить метрики качества (accuracy, precision, recall, F1-score) и построить матрицу ошибок.
7. Сравнить полученные метрики двух моделей и сформулировать выводы.
8. Оформить решение в виде Jupyter Notebook с комментариями.

Теоретические основы

1. Логистическая регрессия (Logistic Regression)

Несмотря на название, логистическая регрессия — это **алгоритм классификации** (чаще всего бинарной), а не регрессии.

1.1. Как она работает?

Вместо предсказания конкретного значения (0 или 1) логистическая регрессия предсказывает **вероятность** принадлежности к классу 1 - значение от 0 до 1. Например, если модель выдаёт 0.932 для спам-письма, это означает 93,2% уверенности, что письмо является спамом.

Как гарантировать, что выход всегда находится между 0 и 1? Для этого используется **сигмоидная функция** (логистическая функция):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

При увеличении входного сигнала звыход приближается к 1, но никогда не достигает его. При уменьшении звыход приближается к 0, но никогда не достигает его.

На вход сигмоидной функции подаётся выход линейной модели:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Таким образом, итоговая формула логистической регрессии:

$$P(y = 1 | X) = \frac{1}{1 + e^{-(wX+b)}}$$

1.2. Ключевые параметры scikit-learn

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    C=1.0,                # обратный коэффициент регуляризации (чем меньше C, тем
                           сильнее регуляризация)
    penalty='l2',         # тип регуляризации (l2, l1, elasticnet)
    solver='lbfgs',       # алгоритм оптимизации
    max_iter=100,         # максимальное число итераций
    random_state=42       # для воспроизводимости
)
```

Параметр **C** контролирует, насколько сильно модель стремится подстроиться под обучающие данные. Маленькое **C** увеличивает регуляризацию, помогая бороться с переобучением.

2. Деревья решений (Decision Trees)

Дерево решений - один из самых наглядных и интерпретируемых алгоритмов машинного обучения. Оно строит модель в виде дерева, где внутренние узлы содержат вопросы («признак > порога?»), листья — предсказания.

2.1. Как строится дерево?

Алгоритм ищет признак и порог, которые **наилучшим образом разделяют** данные. Для этого используются критерии качества разбиения:

- **Энтропия (Entropy)** — мера неопределённости в данных. Если выборка содержит объекты только одного класса, энтропия равна 0 (максимальная «чистота»). Если объекты распределены поровну между двумя классами, энтропия максимальна и приближается к 1.
- **Информационный выигрыш (Information Gain)** — это разница между энтропией родительского узла и средневзвешенной энтропией дочерних узлов. Чем больше информационный выигрыш, тем лучше признак разделяет данные. Дерево решений выбирает тот признак, который даёт максимальный информационный выигрыш.
- **Критерий Джини (Gini Impurity)** — альтернативная мера «нечистоты», часто используется по умолчанию и даёт результат, близкий к энтропии.

2.2. Проблема переобучения и способы борьбы

Дерево решений легко переобучается: если не ограничивать его рост, оно может «выучить» наизусть обучающие данные, но будет плохо работать на новых данных.

Ключевые гиперпараметры для ограничения роста дерева в scikit-learn:

Гиперпараметр	Назначение
max_depth	Максимальная глубина дерева. Если не задать (None), дерево растёт до полной чистоты. Рекомендуется задавать значения от 3 до 10
min_samples_split	Минимальное количество образцов, необходимое для разбиения внутреннего узла. Увеличение этого параметра помогает предотвратить переобучение
min_samples_leaf	Минимальное количество образцов в листовом узле

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion='gini',          # критерий качества ('gini' или 'entropy')
    max_depth=5,              # ограничиваем глубину
    min_samples_split=10,     # требуем минимум 10 образцов для разбиения
    min_samples_leaf=5,       # минимум 5 образцов в листе
    random_state=42
)
```

2.3. Визуализация дерева

Scikit-learn предоставляет возможность визуализировать построенное дерево:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(20,10))
plot_tree(model, feature_names=feature_names, class_names=class_names,
filled=True)
plt.show()
```

3. Метрики оценки качества классификации

Чтобы объективно сравнить модели, недостаточно одной точности, особенно когда классы несбалансированны. Рассмотрим ключевые метрики.

3.1. Матрица ошибок (Confusion Matrix)

Прогнозируемый класс	Реальный класс: Положительный (1)	Реальный класс: Отрицательный (0)
Положительный (1)	TP (True Positive)	FP (False Positive)
Отрицательный (0)	FN (False Negative)	TN (True Negative)

Матрица ошибок позволяет наглядно увидеть, какие ошибки (ложноположительные или ложноотрицательные) преобладают.

3.2. Метрики для бинарной классификации

Метрика	Формула	Что измеряет	Когда важна
Accuracy (точность)	$\frac{TP + TN}{TP + TN + FP + FN}$	Доля правильных ответов	При сбалансированных классах
Precision (точность предсказания положительного класса)	$\frac{TP}{TP + FP}$	Из всех объектов, отнесённых моделью к положительному классу, сколько действительно положительных	Когда важна минимизация ложных срабатываний (например, в медицине — редко пропустить заболевание, даже ценой ложных тревог)
Recall (полнота)	$\frac{TP}{TP + FN}$	Из всех реальных положительных объектов, сколько модель обнаружила	Когда важно найти все положительные объекты (например, когда плата за ложную тревогу невелика, а каждый пропуск случая критичен)
F1-score	$2 \cdot \frac{precision \cdot recall}{precision + recall}$	Среднее гармоническое precision и recall	Для общего сравнения моделей с учётом баланса между precision и recall

3.3. Как интерпретировать `classification_report`

Функция `classification_report` выводит для каждого класса (включая средние):

	precision	recall	f1-score	support
0	0.95	0.98	0.96	200
1	0.97	0.93	0.95	200
accuracy			0.96	400
macro avg	0.96	0.95	0.96	400
weighted avg	0.96	0.96	0.96	400

- **support** — количество объектов данного класса в тестовой выборке.
- **macro avg** — простое среднее по классам.
- **weighted avg** — среднее с весами, пропорциональными support.

Пример 1: Логистическая регрессия на датасете «Рак груди»

```
# Шаг 1: Импорт библиотек
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.metrics import confusion_matrix, classification_report

# Шаг 2: Загрузка данных
cancer = load_breast_cancer()
X = cancer.data          # матрица признаков (569, 30)
y = cancer.target        # целевая переменная (0 — злокачественная, 1 —
доброкачественная)
feature_names = cancer.feature_names
target_names = cancer.target_names

print(f"Размер данных: {X.shape}")
print(f"Классы: {target_names}")
print(f"Распределение классов:\n{pd.Series(y).value_counts()}")

# Шаг 3: Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Шаг 4: Масштабирование признаков (важно для логистической регрессии!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Шаг 5: Обучение логистической регрессии
model_lr = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
model_lr.fit(X_train_scaled, y_train)
```

```

# Шаг 6: Предсказание и оценка
y_pred_lr = model_lr.predict(X_test_scaled)

accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print("=== Логистическая регрессия ===")
print(f"Accuracy: {accuracy_lr:.3f}")
print(f"Precision: {precision_lr:.3f}")
print(f"Recall: {recall_lr:.3f}")
print(f"F1-score: {f1_lr:.3f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr, target_names=target_names))

# Матрица ошибок
cm_lr = confusion_matrix(y_test, y_pred_lr)
plt.figure(figsize=(6,5))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Матрица ошибок — Логистическая регрессия')
plt.show()

```

Ожидаемый результат: Ассурасу ~0.97–0.98, матрица ошибок показывает несколько FP и FN.

Пример 2: Дерево решений на датасете «Ирисы Фишера»

```

# Шаг 1: Импорт библиотек
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Шаг 2: Загрузка данных
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
class_names = iris.target_names

print(f"Размер данных: {X.shape}")
print(f"Классы: {class_names}")

# Шаг 3: Разделение на выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Шаг 4: Обучение дерева решений с ограничением глубины
model_dt = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    min_samples_split=5,
    random_state=42
)
# ограничиваем глубину
# минимум 5 образцов для разбиения

model_dt.fit(X_train, y_train)

# Шаг 5: Предсказание
y_pred_dt = model_dt.predict(X_test)

```

```

# Шаг 6: Оценка качества
accuracy_dt = accuracy_score(y_test, y_pred_dt)

print("=== Дерево решений ===")
print(f"Accuracy: {accuracy_dt:.3f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt, target_names=class_names))

# Шаг 7: Визуализация дерева
plt.figure(figsize=(16, 10))
plot_tree(model_dt, feature_names=feature_names,
          class_names=class_names, filled=True, rounded=True)
plt.title('Дерево решений для классификации ирисов (max_depth=3)')
plt.show()

# Шаг 8: Исследование влияния глубины дерева
depths = range(1, 11)
train_scores = []
test_scores = []

for depth in depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)
    train_scores.append(dt.score(X_train, y_train))
    test_scores.append(dt.score(X_test, y_test))

plt.figure(figsize=(10, 6))
plt.plot(depths, train_scores, label='Точность на обучении', marker='o')
plt.plot(depths, test_scores, label='Точность на тесте', marker='s')
plt.xlabel('Глубина дерева (max_depth)')
plt.ylabel('Точность (Accuracy)')
plt.legend()
plt.title('Зависимость точности от глубины дерева')
plt.grid(True)
plt.show()

```

Ожидаемый результат: при малой глубине (1–2) модель недообучается (точность на тесте низкая), при большой глубине (8–10) появляется переобучение (разрыв между обучением и тестом). Оптимальная глубина - 3–4.

Пример 3: Сравнение двух моделей на одном датасете

```

# Сравнение моделей на датасете load_wine

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

wine = load_wine()
X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Масштабирование для логистической регрессии
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

# Логистическая регрессия
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

# Дерево решений
dt = DecisionTreeClassifier(max_depth=4, min_samples_split=5,
random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

# Сравнение метрик
print("=== Сравнение моделей ===")
print(f"{'Метрика':<15} {'Логистическая регрессия':<25} {'Дерево
решений':<20}")
print("-" * 60)
print(f"{'Accuracy':<15} {accuracy_score(y_test, y_pred_lr):<25.3f}
{accuracy_score(y_test, y_pred_dt):<20.3f}")
print(f"{'Precision (macro)':<15} {precision_score(y_test, y_pred_lr,
average='macro'):<25.3f} "
      f"{precision_score(y_test, y_pred_dt, average='macro'):<20.3f}")
print(f"{'Recall (macro)':<15} {recall_score(y_test, y_pred_lr,
average='macro'):<25.3f} "
      f"{recall_score(y_test, y_pred_dt, average='macro'):<20.3f}")
print(f"{'F1 (macro)':<15} {f1_score(y_test, y_pred_lr,
average='macro'):<25.3f} "
      f"{f1_score(y_test, y_pred_dt, average='macro'):<20.3f}")

```

Варианты для самостоятельного выполнения (10 вариантов)

Для каждого варианта студент должен:

1. Загрузить указанный датасет.
2. Выполнить разбиение на обучающую (80%) и тестовую (20%) выборки с `random_state=42` и `stratify=y`.
3. Построить логистическую регрессию и дерево решений.
4. Для дерева решений подобрать ограничивающие гиперпараметры (`max_depth`, `min_samples_split`) для борьбы с переобучением.
5. Вычислить метрики: accuracy, precision, recall, F1-score (для многоклассовой классификации использовать `average='macro'`).
6. Построить матрицы ошибок для обеих моделей.
7. Сравнить результаты и сделать вывод, какая модель лучше для данного датасета и почему.
8. Оформить решение в Jupyter Notebook.

Вариант 1: Рак груди (бинарная классификация)

- **Датасет:** `load_breast_cancer()` из `sklearn.datasets`
- **Цель:** Классификация опухолей как злокачественной или доброкачественной.
- **Особенность:** Сравнить качество моделей при использовании масштабирования признаков (`StandardScaler`) и без него. Сделать вывод о влиянии масштабирования на логистическую регрессию и дерево решений.

Вариант 2: Ирисы Фишера (многоклассовая классификация, 3 класса)

- **Датасет:** `load_iris()` из `sklearn.datasets`
- **Цель:** Классификация сорта ириса по 4 признакам.
- **Особенность:** Визуализировать построенное дерево решений с помощью `plot_tree` и интерпретировать правила, по которым дерево классифицирует цветы.

Вариант 3: Вина (многоклассовая классификация, 3 класса)

- **Датасет:** `load_wine()` из `sklearn.datasets`
- **Цель:** Определение сорта вина по химическим характеристикам (13 признаков).
- **Особенность:** Попробовать разные критерии качества для дерева решений (`criterion='gini'` и `criterion='entropy'`). Сравнить, влияет ли выбор критерия на точность.

Вариант 4: Цифры (многоклассовая классификация, 10 классов)

- **Датасет:** `load_digits()` из `sklearn.datasets` (8×8 изображения рукописных цифр)
- **Цель:** Распознавание рукописной цифры (0–9) по 64 признакам.
- **Особенность:** Исследовать влияние параметра C (регуляризация логистической регрессии) на точность. Перебрать $C = [0.01, 0.1, 1, 10, 100]$, для каждого C обучить модель и построить график точности от C .

Вариант 5: Рак груди с добавлением шума

- **Датасет:** `load_breast_cancer()`
- **Цель:** Классификация опухоли.
- **Особенность:** Добавить случайный шум в обучающие данные: создать копию `X_train` и добавить к ней `np.random.normal(0, sigma, X_train.shape)`. Исследовать, как увеличение `sigma` (0, 0.1, 0.5, 1.0) влияет на качество логистической регрессии и дерева решений. Какая модель более устойчива к шуму?

Вариант 6: Диабет (задача регрессии) — замена на классификацию!

- **Датасет:** `load_diabetes()` из `sklearn.datasets` (целевая переменная непрерывная, её нужно преобразовать в бинарную)
- **Цель:** Создание бинарной классификации: разделить целевую переменную на две группы (ниже и выше медианы).
- **Особенность:** После разделения выполнить полный цикл классификации: логистическая регрессия против дерева решений. Сравнить, какой алгоритм лучше справляется с задачей прогнозирования «высокого» уровня прогрессирования диабета.

Вариант 7: Кластеризация как предобработка (игрушечный датасет)

- **Датасет:** `make_classification()` из `sklearn.datasets` (можно `make_classification(n_samples=1000, n_features=10, n_classes=3, random_state=42)`)
- **Цель:** Сравнить качество классификации при использовании только исходных признаков и при добавлении признаков, полученных с помощью кластеризации.

- **Особенность:** Создать новый признак — метку кластера (с помощью `KMeans(n_clusters=3)`), добавить в X и посмотреть, улучшилось ли качество моделей.

Вариант 8: Несбалансированные классы

- **Датасет:** `make_classification(n_samples=1000, n_features=10, weights=[0.9, 0.1], random_state=42)`
- **Цель:** Классификация с сильным дисбалансом классов.
- **Особенность:** Использовать `class_weight='balanced'` в логистической регрессии и оценить, как это влияет на `precision` и `recall` редкого класса по сравнению с моделью без балансировки.

Вариант 9: Временные ряды — классификация тренда

- **Датасет:** создать синтетические временные ряды: один тип — возрастающий, другой — убывающий, третий — стационарный (по 200 образцов каждого типа)
- **Цель:** Классификация формы временного ряда по агрегированным статистикам (среднее, дисперсия, коэффициент наклона линейной регрессии).
- **Особенность:** Выполнить предобработку (создание признаков из временных рядов) и сравнить, какая модель (логистическая регрессия или дерево) лучше различает три типа трендов.

Вариант 10: Искусственные данные с нелинейной границей

- **Датасет:** `make_moons(n_samples=500, noise=0.1, random_state=42)` и `make_circles(n_samples=500, noise=0.05, random_state=42)`
- **Цель:** Классификация объектов на двух наборах с нелинейной формой границы (два полумесяца и концентрические окружности).
- **Особенность:** Сравнить логистическую регрессию (линейная граница) и дерево решений (нелинейная граница) на этих двух типах данных. Визуализировать границы решений и сделать вывод о применимости каждого алгоритма.

Контрольные вопросы для самопроверки

1. Почему логистическая регрессия называется «регрессией», хотя используется для классификации?
Ответ: Потому что внутри неё сначала вычисляется линейная комбинация признаков (как в регрессии), а затем результат преобразуется в вероятность с помощью сигмоидной функции.
2. Что такое сигмоидная функция и зачем она нужна в логистической регрессии?
Ответ: Сигмоидная функция преобразует любое действительное число в интервал $(0, 1)$, что позволяет интерпретировать выход модели как вероятность.
3. В чём заключается проблема переобучения у дерева решений? Как с ней бороться?
Ответ: Переобучение возникает, когда дерево слишком точно подстраивается под обучающие данные и начинает «запоминать» шум. Способы борьбы: ограничение глубины (`max_depth`), увеличение `min_samples_split` и `min_samples_leaf`.

4. В каком случае **precision** больше подходит для оценки модели, чем **recall**?
Приведите пример.
Ответ: Precision важен, когда ложные положительные срабатывания имеют высокую стоимость. Например, в медицинском скрининге, если модель ошибочно назначает дорогостоящее и инвазивное лечение здоровому пациенту, каждый ложный положительный случай критичен — здесь важна высокая precision.
5. Что показывает **F1-score** и почему он считается «балансирующей» метрикой?
Ответ: F1-score — это гармоническое среднее между precision и recall. Он принимает высокие значения только в том случае, если оба показателя высоки, и может служить единой метрикой для сравнения моделей.
6. Почему для многоклассовой классификации используют **average='macro'** (и как оно рассчитывается)?
Ответ: average='macro' вычисляет среднее арифметическое показателей (precision/recall) по всем классам, без учёта их размера. Это даёт одинаковую важность каждому классу, даже если они несбалансированы.
7. В чём разница между энтропией и критерием Джини при построении дерева решений?
Ответ: Оба критерия измеряют «нечистоту» узла. Энтропия основана на теории информации, критерий Джини — на вероятностных оценках. На практике они дают схожие результаты, но Джини считается немного быстрее для вычислений.
8. При каком условии можно использовать **logistic regression** для многоклассовой классификации?
Ответ: Существует два подхода: OvR (one-vs-rest) — для каждого класса обучается отдельный бинарный классификатор; и multinomial (softmax) — модель предсказывает вероятности для всех классов одновременно.
9. Как параметры **max_depth** и **min_samples_split** влияют на сложность модели?
Ответ: Увеличение max_depth и уменьшение min_samples_split делают модель более сложной (способной улавливать мелкие детали), но повышают риск переобучения. Уменьшение max_depth и увеличение min_samples_split, напротив, упрощают модель.
10. Что означают термины «ложноположительный» и «ложноотрицательный» результат? Приведите пример.
Ответ: Ложноположительный (FP) — «предсказано, что объект принадлежит положительному классу, хотя на самом деле нет». Ложноотрицательный (FN) — «предсказано, что объект не принадлежит положительному классу, хотя на самом деле принадлежит». Например, в задаче определения спама: FP — письмо без спама помечено как спам, FN — спам не обнаружен.

Дополнительные источники информации и литература

1. Официальная документация **scikit-learn** (логистическая регрессия):
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
2. Официальная документация **scikit-learn** (дерева решений):
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
3. Объяснение логистической регрессии простыми словами (Google для разработчиков):
<https://developers.google.com/machine-learning/crash-course/logistic-regression>

4. Деревья решений — понятным языком ([Habr.com](https://habr.com/ru/articles/151164/)):
<https://habr.com/ru/articles/151164/>
5. Метрики качества в задачах классификации — статья на Habr:
<https://habr.com/ru/companies/ods/articles/328372/>
6. Книга «Python и анализ данных» (Уэс Маккинни) — глава про классификацию.
7. Курс «Машинное обучение» на Stepik (разделы по деревьям и логистической регрессии):
<https://stepik.org/course/4852>
8. Сравнение логистической регрессии и деревьев решений — KDnuggets:
<https://www.kdnuggets.com/2020/12/comparison-logistic-regression-decision-tree.html>

Критерии оценивания (100-балльная шкала)

№	Критерий	Баллы	Комментарий
1.	Загрузка и первичный анализ данных	8	Загрузка датасета, вывод <code>.shape</code> , <code>.info()</code> , <code>.head()</code> , распределение классов
2.	Разделение на X, y и выборки (<code>train_test_split</code>)	8	Правильное выделение признаков и целевой переменной; использование <code>stratify=y</code> и <code>random_state</code>
3.	Логистическая регрессия — обучение и предсказание	15	Корректные <code>fit()</code> и <code>predict()</code> ; при необходимости — масштабирование признаков
4.	Дерево решений — обучение и предсказание	15	Корректное создание и обучение модели; установка гиперпараметров для борьбы с переобучением
5.	Вычисление метрик качества	20	Вычислены accuracy, precision, recall, F1-score для обеих моделей; для многоклассовой — <code>average='macro'</code>
6.	Построение и интерпретация матрицы ошибок	12	Построены матрицы ошибок для обеих моделей; сделан краткий анализ (каких ошибок больше, FP или FN)
7.	Сравнительный анализ моделей	12	Написан содержательный вывод: какая модель лучше, почему, в чём сильные и слабые стороны
8.	Оформление кода и блокнота	10	Комментарии к ключевым шагам, логическая структура ячеек, Markdown-заголовки, чистый вывод
9.	Использование системы контроля версий Git (<code>push</code> на GitHub)	+5	Бонусные баллы за загрузку работы в удалённый репозиторий
	ИТОГО	100+5	

Шкала перевода баллов в оценку

- **90–100 (105 с бонусом) баллов** — отлично (все этапы выполнены корректно, глубокий анализ, качественные визуализации)
- **75–89 баллов** — хорошо (небольшие недочёты, например, не проведено сравнение всех метрик)
- **60–74 балла** — удовлетворительно (пропущен один из этапов или допущены существенные ошибки)

- **менее 60 баллов** — неудовлетворительно (требуется повторное выполнение)